WEBRUM-04: Session Analysis

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 4 of 10 | **Created:** March 2026 | **Last Updated:** 10/31/2026

Overview

User session analysis is the foundation of understanding how real users interact with your web applications. A session represents a complete visit — from the first page load to the last interaction before timeout. By analyzing sessions, you can identify user journey patterns, measure engagement, track conversions, calculate bounce rates, and understand how geographic location and device type impact the user experience.

Table of Contents

1. Session Properties
2. Session Segmentation
3. User Journey Mapping
4. Conversion Tracking
5. Bounce Rate Analysis
6. Geographic Analysis
7. Device and Browser Analysis
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with session capture active
Session Properties	Custom session properties configured (optional but recommended)
Permissions	`storage:user.events:read`, `storage:user.sessions:read`
Previous Notebook	WEBRUM-01: RUM Fundamentals

1. Session Properties

Dynatrace automatically captures standard session properties and allows you to define custom ones:

Standard Properties

Property	Description	Example Values
`dt.rum.session.id`	Unique session identifier	`23626166142035610_1-0`
`dt.rum.user_type`	Session classification	`real_user`, `robot`, `synthetic`
`start_time` / `end_time`	Session start and end — `user.sessions` has no populated `timestamp`	`2026-10-05T15:18:09Z`
`duration`	Total session duration	`300000000000` (nanoseconds)
`user_action_count`	Number of user actions	`15`
`navigation_count`	Number of navigations	`4`
`error.count`	Number of errors in session	`2`

Property	Description	Example Values
`geo.country.iso_code`	User's country	`US`
`os.name`	Operating system	`Windows`, `macOS`, `iOS`
`browser.name`	Browser	`Chrome`, `Firefox`, `Safari`
`browser.window.width`	Browser window width in pixels	`1920`
`browser.window.height`	Browser window height in pixels	`1080`

These are the New RUM names the queries below use. The classic names (`sessionId`, `userType`, `userActionCount`, `totalErrorCount`, `osFamily`, `browserFamily`) read null on New RUM data, so a query copied with them returns nothing and raises no error. Web RUM has no connection-type field: `network.connection.type` is documented as *"Only supported by OneAgent for Mobile."*

> **Sources:** User events — semantic dictionary (DT docs) — *"The internet connection type. Only supported by OneAgent for Mobile."* **Dictionary:** model `rum.user_session` lists `start_time`, `end_time`, `duration`, `user_action_count`, `navigation_count`, `error.count`, `dt.rum.session.id`, `dt.rum.user_type`, `geo.country.iso_code`, `os.name`, `browser.name`, `browser.window.width` / `.height` and no `timestamp`, read 10/05/2026.

Custom Session Properties

Define custom properties via **Settings > Web and mobile monitoring > Session and user action properties**. Common examples:

- `session_properties.user_id` — Authenticated user identifier
- `session_properties.plan_type` — Subscription tier (free, pro, enterprise)
- `session_properties.cart_value` — Shopping cart total
- `session_properties.ab_variant` — A/B test variant

> **Sources:** User sessions — semantic dictionary (DT docs) — *"Contains the value for the session property named"*.

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; frontend.name (stable; an array on user.sessions, so
expand it before grouping)
//                  primary_tags.application exists only if your OpenPipeline
adds it (corrected 10/14/2026)
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.value (page summaries;
web_vitals.time_to_first_byte where populated) —
//                  NOT ttfb.waiting_duration, which is the pre-request
wait/redirect phase
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the documented values are "real_user" / "robot" / "synthetic"
(lowercase);
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Explore session data — view available fields
fetch user.sessions, from:-1h
| filter dt.rum.user_type == "real_user"
| limit 5
```

2. Session Segmentation

Segmenting sessions helps identify patterns across different user groups. Common segmentation dimensions include engagement level, error impact, and return frequency.

```
// Engagement segmentation — bucket sessions by number of actions. Sessions
with zero user actions
// get their own bucket; they used to fall into "Low (2-3 actions)" (corrected
10/05/2026).
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| fieldsAdd engagement = if(user_action_count == 0, "No actions",
    else: if(user_action_count == 1, "Bounce",
    else: if(user_action_count &lt;= 3, "Low (2-3 actions)",
    else: if(user_action_count &lt;= 10, "Medium (4-10 actions)",
    else: "High (10+ actions)"))))
| summarize {session_count = count(),
    avg_duration = avg(duration),
    avg_errors = avg(error.count)},
    by:{engagement}
| sort session_count desc
```

```
// Error-impacted sessions — how many sessions had errors?
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize {total_sessions = count(),
    error_sessions = countIf(error.count &gt; 0)},
    by:{frontend.name}
| fieldsAdd error_session_pct = round(toDouble(error_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort error_session_pct desc
```

3. User Journey Mapping

Understanding the sequence of actions within sessions reveals common navigation paths, dead ends, and drop-off points.

```
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**,
but was written
// against names that are null on New RUM data, so these cells returned
nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
javascript agent):
//   action.type == "Load"             -&gt; characteristics.has_page_summary
== true
//                                       (was characteristics.classifier —
"not intended for query usage", SD 1.349)
//   action.type                       -&gt; user_action.type
(hard_navigation | soft_navigation | same_view | api)
//   action.name                       -&gt; page.detected_name
//   LCP / CLS                         -&gt;
web_vitals.largest_contentful_paint / web_vitals.cumulative_layout_shift
//                                       (stable — the fields Dynatrace's own
DQL examples use), falling back to
//                                       the experimental lcp.start_time /
cls.value (corrected 10/14/2026)
//   app.name                          -&gt; frontend.name (stable; an array
on user.sessions)
// UNITS DIFFER BETWEEN THE TWO FAMILIES. web_vitals.largest_contentful_paint
is a DURATION, so
// `/ 1ms` turns it into milliseconds; lcp.start_time is a PLAIN NUMBER
already in milliseconds,
// and dividing it by 1ms yields null. The CWV cells therefore read
// coalesce(web_vitals.largest_contentful_paint / 1ms, lcp.start_time) and
// coalesce(web_vitals.cumulative_layout_shift, cls.value), against the
2500/4000 ms and 0.1/0.25 thresholds.
// Why both: on the validation tenant every user event comes from the
synthetic engine, which fills
// lcp.* / cls.* but not web_vitals.* (0 of 243,494 page summaries in 30 days,
10/14/2026). That
// is a property of synthetic-monitor data, not a sign that web_vitals.* is
rarely populated.
// Top entry pages — where do users start their journey?
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| summarize first_action = takeFirst(page.detected_name), by:
{dt.rum.session.id}
| summarize entry_count = count(), by:{first_action}
| sort entry_count desc
| limit 10
```

```
// Top exit pages — where do users leave?
fetch user.events, from:-24h
| filter characteristics.has_page_summary == true
| summarize last_action = takeLast(page.detected_name), by:{dt.rum.session.id}
| summarize exit_count = count(), by:{last_action}
| sort exit_count desc
| limit 10
```

```
// Session duration distribution by hour of day — when are users most active?
fetch user.sessions, from:-7d
| filter dt.rum.user_type == "real_user"
| fieldsAdd hour = getHour(start_time)   // user.sessions has no populated
timestamp
| summarize {session_count = count(),
    avg_duration_sec = avg(duration / 1s)},
    by:{hour}
| sort hour asc
```

4. Conversion Tracking

Conversion tracking identifies which sessions completed a desired business action (purchase, signup, form submission). This requires either:

- **Session properties** marking conversion events
- **Specific user actions** that indicate conversion (e.g., a "Thank You" page load)

Conversion via Action Name Detection

If your conversion page has a recognizable URL pattern, you can detect conversions from user actions:

```
// Conversion rate — share of real-user sessions that reached a
checkout/confirmation page.
// Adapt the page-name tests to match your conversion page URL pattern.
// Computed from the events side alone (corrected 10/05/2026): the previous
version appended two
// unrelated row sets and never produced a rate.
fetch user.events, from:-24h
| filter dt.rum.user_type == "real_user"
| filter characteristics.has_page_summary == true
| fieldsAdd is_conversion_page = contains(page.detected_name, "confirmation")
or contains(page.detected_name, "thank-you") or contains(page.detected_name,
"checkout-success")
| summarize {conversion_views = countIf(is_conversion_page)}, by:
{dt.rum.session.id}
| summarize {sessions = count(), converted_sessions = countIf(conversion_views
&gt; 0)}
| fieldsAdd conversion_rate_pct = round(toDouble(converted_sessions) /
toDouble(sessions) * 100.0, decimals: 2)
```

> **Tip:** For accurate conversion tracking, use Dynatrace session properties to flag conversion events. This avoids reliance on URL pattern matching, which can be fragile.

5. Bounce Rate Analysis

A "bounce" is a session with only one user action — the user loaded one page and left. High bounce rates may indicate poor landing page relevance, slow performance, or broken functionality.

Dynatrace's own definition: *"A session with only one user action or navigation is tagged as bounced."* The queries in this series count a bounce as `user_action_count == 1`, the same rule in WEBRUM-04, -05, -08 and -09. Sessions with zero user actions are kept out of the bounce count and shown separately in the engagement query above. Dynatrace's definition also counts a session whose only activity is a single navigation, so compare the result with the **Bounced Sessions** filter in Users & Sessions on your tenant before treating the two as identical.

> **Sources:** User sessions in web frontends (DT docs) — *"A session with only one user action or*

> *navigation is tagged as bounced."*

```
// Bounce rate by application
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize {total_sessions = count(),
    bounced_sessions = countIf(user_action_count == 1)},
    by:{frontend.name}
| fieldsAdd bounce_rate_pct = round(toDouble(bounced_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort bounce_rate_pct desc
```

```
// Bounce rate trend over 7 days — track improvement over time
fetch user.sessions, from:-7d
| filter dt.rum.user_type == "real_user"
| fieldsAdd is_bounce = if(user_action_count == 1, 1, else: 0)
| makeTimeseries {total = count(), bounces = sum(is_bounce)}, interval:24h
| fieldsAdd bounce_rate = bounces[] * 100.0 / total[]   // per-day rate, not
one collapsed number
```

6. Geographic Analysis

RUM data includes geographic information derived from the user's IP address. This helps identify regional performance differences and target optimization efforts.

```
// Sessions by country — top 15 countries by volume
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(geo.country.name)
| summarize {session_count = count(),
    avg_actions = avg(user_action_count),
    avg_errors = avg(error.count),
    avg_duration_sec = avg(duration / 1s)},
    by:{geo.country.name}
| sort session_count desc
| limit 15
```

```
// Top 10 cities by session count with average performance
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(geo.city.name) and isNotNull(geo.country.name)
| summarize session_count = count(), by:{geo.city.name, geo.country.name}
| sort session_count desc
| limit 10
```

7. Device and Browser Analysis

Understanding the device and browser mix helps prioritize testing and optimization efforts.

```
// Session distribution by browser
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(browser.name)
| summarize {session_count = count(),
    avg_errors = avg(error.count)},
    by:{browser.name}
| sort session_count desc
| limit 10
```

```
// Session distribution by operating system
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(os.name)
| summarize {session_count = count(),
    avg_duration_sec = avg(duration / 1s)},
    by:{os.name}
| sort session_count desc
```

```
// Screen resolution distribution — identify common viewport sizes
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(browser.window.width) and isNotNull(browser.window.height)
| fieldsAdd resolution = concat(toString(browser.window.width), "x",
toString(browser.window.height))
| summarize session_count = count(), by:{resolution}
| sort session_count desc
| limit 10
```

8. Summary and Next Steps

In this notebook, we covered:

- **Session properties** — Standard and custom properties for segmentation
- **Engagement segmentation** — Bucketing sessions by interaction depth
- **User journey mapping** — Entry/exit pages and activity by time of day
- **Conversion tracking** — Identifying sessions that completed business goals
- **Bounce rate analysis** — Measuring and trending single-action sessions
- **Geographic analysis** — Session distribution by country and city
- **Device/browser analysis** — Understanding the technology mix of your users

Next Steps

- **WEBRUM-05: Error Analysis** — JavaScript error tracking and impact analysis
- **WEBRUM-06: Performance Analysis** — Deep dive into page load waterfall timings

References

- Define user action and session properties (DT docs)
- USQL — custom session queries (DT docs)
- User events — semantic dictionary (DT docs) — *"Used for internal optimization when storing the data and not intended for query usage."*
- Semantic Dictionary changelog 1.349 (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.